



Projet : 30 %

Identification du cours

Nom du (des) programme(s) – Code(s) : TECHNOLOGIE DE L'INFORMATIQUE –
INTELLIGENCE ARTIFICIELLE ET APPRENTISSAGE
AUTOMATIQUE – LEA.DQ

Titre du cours : INTRODUCTION À LA PROGRAMMATION

Numéro du cours : 420-A03-AS

Groupe : 05421

Nom de l'enseignante: Asma Aouichat

Durée de l'évaluation : 3 périodes (150 minutes)

Session : Automne 2025

Identification de l'étudiant

Nom : _____ Numéro d'étudiant : _____

Date : _____ Résultat : _____

☐ Je déclare que ceci est un travail original et que j'ai mentionné toutes les sources du contenu dont je ne suis pas l'auteur (sources en ligne et imprimées, images, graphiques, films, etc.) dans le style de citation requis.

Standard de la (des) compétence(s) évaluée(s)

Énoncé de la (des) compétence(s) évaluée(s) – Code(s)

- Utiliser des langages de programmation– KP48
- Développer des programmes– KP55

Éléments de la (des) compétence(s) évalués

- Pour des problèmes qui peuvent être automatisés
- Pour des problèmes situationnels tels que présentés
- À l'aide d'algorithmes
- À l'aide d'un débogueur et d'un plan de tests fonctionnels
- Travailler dans différents types de milieux de travail
- À partir de tâches ou d'un algorithme prédéfini(es)
- À partir d'un modèle de données
- Utilisation du logiciel et des documents de référence appropriés
- Respect du contrat de licence

Consignes

- Les notes de cours et le dictionnaire sont permis.
 - Aucune pause ne sera accordée pendant l'examen. Aucun étudiant ne peut quitter la salle d'examen avant que la moitié du temps alloué ne se soit écoulée. L'étudiant qui quitte la salle d'examen ne peut y revenir (PIEA – Article 5.12.4).
 - L'enseignant(e) ne répondra pas aux questions.
 - Les étudiants ne sont pas autorisés à communiquer entre eux.
 - L'enseignant(e) a le devoir d'identifier les erreurs de langue dans les travaux des étudiants. Un pourcentage attribué à la langue va jusqu'à 20 % de la note (PIEA – Article 5.7).
 - Le plagiat, la tentative de plagiat ou la coopération à un plagiat lors d'une épreuve sommative entraîne la note zéro (0). Dans le cas de récidive, dans le même cours ou dans un autre cours, l'étudiant se voit octroyer un « 0 » pour le cours concerné. (PIEA – Article 5.16).
 - Veuillez écrire de façon lisible.
-

Répartition des points

Cette évaluation est sur 100 points répartis comme suit :

- | | |
|----------------------------------|----------------------------|
| • Iteration 1(Mardi 28 octobre) | Pour un total de 10 points |
| • Iteration 2(Mardi 4 novembre) | Pour un total de 14 points |
| • Iteration 3(Mardi 11 novembre) | Pour un total de 14 points |
| • Iteration 4(Mardi 18 novembre) | Pour un total de 14 points |
| • Iteration 5(Mardi 25 novembre) | Pour un total de 14 points |
| • Iteration 6(Mardi 2 décembre) | Pour un total de 14 points |
| • Iteration 7(Mardi 9 décembre) | Pour un total de 14 points |
| TOTAL: | 100 POINTS |
-

I. Context général :

L'objectif central de ce projet est de créer une application fonctionnelle en suivant une méthode itérative, permettant de progresser graduellement à travers les différentes étapes du développement logiciel. Ce projet vous offre l'opportunité de développer vos compétences en analyse des besoins, en conception d'algorithmes, en programmation rigoureuse en C++, ainsi qu'en tests et en documentation du code. Cette approche a également pour but de renforcer vos savoir-faire en gestion de projet, en encourageant le travail collaboratif au sein de petits groupes et en assurant un suivi méthodique grâce à des livrables complets à chaque étape du projet.

Ce projet vise à mobiliser les compétences clés suivantes :

- **Analyse d'un besoin client** et modélisation de données ;
- **Développement structuré** en langage C++ ;
- **Décomposition du problème** en modules et sous-fonctions ;
- **Mise en œuvre d'algorithmes simples à complexes** (boucles, conditions, structures de données) ;
- **Validation et test du code**, avec réflexion sur les cas d'erreurs courants (ex : saisie invalide, choix hors menu) ;
- **Documentation technique** claire pour assurer la maintenabilité du code.

II. Guide de réalisation du projet informatique

1. Constitution des groupes

Les projets se réaliseront en petits groupes de 2 à 3 personnes maximum.

2. Choix du sujet

Chaque groupe devra sélectionner un sujet de projet parmi une liste proposée, ou bien proposer un sujet original sous validation préalable de l'enseignante. Le sujet choisi devra être suffisamment précis et adapté à la portée du module.

3. Organisation du travail par itérations

Le projet sera structuré en **7 itérations** distinctes. Chaque itération correspond à une étape claire avec un objectif spécifique de développement. Cette approche permet une progression maîtrisée, favorise la validation progressive des acquis, et facilite la correction par l'enseignante.

Objectifs des itérations

Chaque itération vise à développer une partie fonctionnelle cohérente du projet, en ajoutant progressivement des fonctionnalités, des améliorations ou des corrections.

4. Modalités de remise du travail pour chaque itération

Chaque itération devra faire l'objet d'une remise complète et structurée, comprenant impérativement les deux éléments suivants :

1. Code source en C++ (version complète)

- Fournir tous les fichiers sources C++ correspondant à l'itération réalisée.
- Le code doit être **clairement commenté**, en particulier pour les structures importantes : conditions (if/else), boucles (for, while), fonctions, structures, etc.
- Le programme doit être **lisible et bien organisé**, avec une indentation correcte, des noms de variables explicites et un agencement logique des blocs de code.

2. Rapport d'itération (document explicatif au format PDF)

Ce document accompagne la remise du code et détaille la démarche de développement. Il doit contenir les parties suivantes :

a) Introduction

- Présentation synthétique de l'objectif principal de l'itération.

b) Fonctionnalités développées

- Description précise des fonctionnalités ajoutées ou modifiées dans cette version.
- Illustration par des exemples concrets ou des scénarios d'utilisation du programme.

c) Algorithmes utilisés

- Présentation claire de la logique de programmation adoptée : utilisation des conditions, boucles, fonctions, structures de données, etc.
- Représentation visuelle ou simplifiée (pseudo-code, diagrammes de flux, tableaux logiques) pour faciliter la compréhension.

d) Problèmes rencontrés et solutions

- Analyse des difficultés techniques, conceptuelles ou organisationnelles rencontrées.
- Description des méthodes et stratégies employées pour surmonter ces obstacles : recherches, tests, aide extérieure, documentation.

e) Pistes d'amélioration

- Propositions d'améliorations possibles pour le code existant.
- Suggestions d'ajouts fonctionnels ou d'optimisations futures.
- Réflexion personnelle sur la qualité et l'évolution du projet.

f) Tests réalisés

- Description des tests effectués (manuels ou automatisés) pour garantir la fiabilité du programme.
- Présentation de scénarios de tests détaillés : données d'entrée, résultats attendus, cas limites.
- Captures d'écran illustrant l'exécution du programme et les résultats obtenus.

5. Plan suggéré pour le rapport d'itération

1. Introduction
2. Fonctionnalités implémentées
3. Algorithmes développés
 - 3.1 Description générale
 - 3.2 Pseudo-code / Diagrammes
4. Difficultés rencontrées et solutions
5. Améliorations envisagées
6. Tests réalisés (avec captures d'écran)
7. Conclusion

6. Recommandations générales

- Respecter les délais de remise pour chaque itération.
- Travailler en équipe en partageant clairement les tâches.
- Documenter le code et le rapport de manière rigoureuse.
- Tester régulièrement le programme pour détecter et corriger les erreurs au plus tôt.

III. Suggestion des projets:

Projet 1. Gestion d'un restaurant (commandes et menu)

Dans le cadre de ce projet de fin de module, nous nous intéressons à la **digitalisation de la gestion des commandes** dans un petit restaurant de type traditionnel. Aujourd'hui, ce restaurant utilise encore un **système manuel à base de carnet papier** pour noter les commandes des clients, calculer le total à payer et suivre les plats proposés. Cette méthode, bien que simple, présente plusieurs limites : erreurs de saisie, perte d'informations, lenteur dans la prise en charge des clients et absence de traçabilité des commandes. Le restaurateur souhaite donc moderniser son système de gestion afin de **gagner en efficacité, en fiabilité et en professionnalisme**.

Le **cas d'étude** concerne un établissement de restauration disposant d'un **menu fixe** composé d'une dizaine de plats (ex : pizza, salade, steak-frites, lasagnes, etc.), chacun étant associé à un prix unique. Le personnel doit pouvoir prendre les commandes rapidement, calculer le total de la commande sans erreur, et garder un historique si besoin. Le projet vise donc à concevoir une **application informatique simple en C++** fonctionnant en mode console.

1.1.Objectifs du projet

L'objectif principal est de **développer une application modulaire** qui facilitera :

- La **consultation du menu** par le personnel ou les clients ;
- La **saisie des commandes** en sélectionnant un ou plusieurs plats et quantités ;
- Le **calcul automatique du total à payer** ;
- La **gestion future des clients** et l'historique des commandes, dans une logique d'évolution progressive.

1.2.Choix techniques

L'application sera développée en **langage C++**, en utilisant une approche **itérative**, permettant d'introduire progressivement des concepts de programmation fondamentaux : entrées/sorties, conditions, boucles, fonctions, tableaux, structures, et plus tard, vecteurs dynamiques.

Le programme fonctionnera **en ligne de commande**, sans interface graphique, mais avec une présentation claire et conviviale (ex : menu numéroté, messages d'erreurs explicites, affichage formaté des factures).

Exemples de scénarios d'utilisation

1. Exemple 1 – Commande simple :

- Le serveur affiche le menu :
 - 1. Pizza Margherita - 8.50 €
 - 2. Lasagnes - 9.00 €
 - 3. Salade César - 7.00 €
 - 4. Steak-frites - 11.00 €
- Le client choisit : 1 pizza et 2 salades.
- L'application calcule automatiquement le total : $1 \times 8.50 + 2 \times 7.00 = \mathbf{22.50 \text{ €}}$.
- Un récapitulatif est affiché avec les quantités et le total dû.

2. Exemple 2 – Commande multiple avec historique (itération 7) :

- Le client "Alice Martin" passe une commande.
- Plus tard, elle revient. Le serveur saisit son nom, et l'application affiche son historique :
- Commande précédente :
 - 2x Lasagnes (18.00 €)
 - 1x Salade César (7.00 €)
- Total : 25.00 €

1.3.Périmètre initial et évolutif

Dans un premier temps (itérations 1 à 5), l'application permettra de :

- Afficher dynamiquement un menu ;
- Gérer des commandes multi-plats avec quantités ;
- Calculer et afficher un total clair ;
- Organiser les données via des structures et tableaux.

Dans un second temps (itérations 6 et 7), des fonctionnalités avancées seront ajoutées :

- **Modification ou annulation d'un plat** dans la commande ;
- Application éventuelle de **remises** (ex : -10 % pour commandes supérieures à 50 €) ;
- **Enregistrement de clients** avec leur nom ;
- **Historique des commandes** associé à chaque client ;
- Utilisation de **structures imbriquées** pour gérer plusieurs entités (clients, commandes, plats).

1.4. Itérations :

1.4.1. Itération 1 — Interaction simple avec l'utilisateur

Créer une première version du programme permettant une interaction basique avec l'utilisateur : afficher un menu statique et prendre une commande simple.

Fonctionnalités développées

- Affichage d'un menu fixe (3 à 5 plats) avec leur prix.
- Saisie d'un choix par l'utilisateur (ex : numéro du plat).
- Validation de la saisie (numéro correct ou non).
- Affichage du plat sélectionné et de son prix.

Exemple

Menu :

1. Pizza Margherita - 8.50 €
2. Salade César - 7.00 €
3. Lasagnes - 9.00 €

Votre choix : 2

Vous avez choisi : Salade César – 7.00 €

Compétences travaillées

- Utilisation des **entrées/sorties standard** (cin, cout).
- Application de **structures conditionnelles** (if/else) pour valider les choix.
- Gestion d'erreurs simples (message si saisie invalide).

Améliorations possibles

- Permettre la saisie de plusieurs plats.
- Ajouter des noms de clients ou des identifiants de commande.

1.4.2. Itération 2 — Commander plusieurs plats

Permettre à l'utilisateur de passer une commande comprenant **plusieurs plats** successifs, avec **calcul du total à payer**.

Fonctionnalités développées

- Possibilité de saisir un plat, puis d'indiquer s'il souhaite en commander un autre.
- Accumulation du montant total des plats choisis.
- Affichage du montant final.

Exemple

Plat 1 : Pizza – 8.50 €

Souhaitez-vous commander un autre plat ? (o/n) : o

Plat 2 : Lasagnes – 9.00 €

Total de votre commande : 17.50 €

Compétences travaillées

- Utilisation de **boucles** (while, do-while) pour répéter la commande.
- **Stockage temporaire** des totaux dans des variables.
- Gestion de la **logique de contrôle utilisateur** (continuer/arrêter).

Améliorations possibles

- Permettre à l'utilisateur de **saisir des quantités** pour chaque plat.
- Enregistrer chaque ligne de commande pour l'afficher à la fin.

1.4.3. Itération 3 — Modularisation avec fonctions

Structurer le code en **fonctions claires et réutilisables** pour améliorer la lisibilité, la maintenance et la séparation des tâches.

Fonctionnalités développées

- Création de plusieurs fonctions :
 - afficherMenu()
 - demanderChoix()
 - calculerTotal()
- Passage de paramètres et récupération de valeurs.

Compétences travaillées

- Définition et appel de **fonctions personnalisées**.
- Utilisation de **paramètres et valeurs de retour**.
- Organisation logique du programme en **blocs fonctionnels**.

Améliorations possibles

- Utiliser une fonction pour afficher le récapitulatif final.
- Créer une fonction pour gérer la validation des entrées.

1.4.4. Itération 4 — Gestion du menu avec tableaux

Rendre le menu **dynamique** en utilisant des tableaux pour stocker les plats et les prix, facilitant ainsi leur gestion.

Fonctionnalités développées

- Deux tableaux parallèles :
 - Un pour les **noms des plats** ;
 - Un pour les **prix**.
- Parcours des tableaux pour afficher le menu.
- Vérification des choix en fonction de l'indice du tableau.

Compétences travaillées

- Déclaration et utilisation de **tableaux statiques**.
- Utilisation de **boucles for** pour afficher le menu.
- **Validation automatique** en fonction de la taille du menu.

Améliorations possibles

- Rendre le menu **modifiable par l'utilisateur** (ajout/suppression).
- Utiliser des **tableaux de structures** au lieu de tableaux parallèles.

1.4.5. Itération 5 — Introduction des structures pour modéliser les plats

Utiliser des **structures (struct)** pour représenter les plats de façon plus claire et évolutive.

Fonctionnalités développées

- Définition d'une struct Plat contenant le nom et le prix.
- Création d'un tableau de Plat pour le menu.
- Enregistrement des **quantités commandées** dans un tableau parallèle.
- Calcul du total pour chaque ligne de commande.

Compétences travaillées

- Définition et manipulation de **structures personnalisées**.
- **Tableaux de structures** et gestion associée.
- Affichage formaté : nom, quantité, prix unitaire, total par plat.

Améliorations possibles

- Afficher un **ticket de caisse final** bien présenté.
- Ajouter des champs comme la **catégorie du plat** (entrée, plat, dessert).

1.4.6. Itération 6 — Modifications, annulations, remises

Donner à l'utilisateur la possibilité de **modifier sa commande**, d'annuler un plat ou d'appliquer des remises automatiques.

Fonctionnalités développées

- Système de modification (changement de quantité).

- Suppression d'un plat déjà sélectionné.
- Application automatique d'une **remise** si seuil atteint (ex : 10 % à partir de 50 €).

Exemple

Votre total est de 52.00 €

Remise appliquée : -10%

Nouveau total : 46.80 €

Compétences travaillées

- Utilisation avancée de **conditions et boucles imbriquées**.
- Gestion des **cas d'annulation ou modification**.
- Calculs dynamiques sur des **totaux avec remise**.

Améliorations possibles

- Interface de commande plus interactive.
- Remises personnalisées selon le client (fidélité).

Itération 7 — Gestion des clients et historique des commandes

Associer les commandes à des **clients nommés** et permettre la **consultation d'un historique** par client.

Fonctionnalités développées

- Demande du **nom du client** au début de la commande.
- Enregistrement de toutes les commandes dans un tableau (ou vecteur) de clients.
- Possibilité de consulter l'historique d'un client à tout moment.

Compétences travaillées

- Structures imbriquées : Client contenant une liste de commandes.
- Manipulation de **tableaux dynamiques (ou vector)**.
- Recherche dans les enregistrements (par nom).

Améliorations possibles

- Export de l'historique en **fichier texte**.
- Ajout d'un **système de points de fidélité** ou de compte client.

Projet 2 : Simulation d'un guichet automatique bancaire (GAB/ATM)

Dans le cadre de ce projet, l'objectif est de simuler un distributeur automatique de billets (GAB ou ATM) qui propose des fonctionnalités basiques mais essentielles pour la gestion d'un compte bancaire. Aujourd'hui, les guichets automatiques sont des outils incontournables qui permettent aux clients d'accéder à leurs comptes rapidement, de manière sécurisée et autonome. Cependant, comprendre les mécanismes internes, comme la vérification d'identité, la gestion des opérations et la sécurisation des transactions, reste un défi pédagogique intéressant pour les étudiants en programmation.

Ce projet s'appuie sur la modélisation informatique d'un système simple de guichet automatique, en s'intéressant particulièrement aux interactions fondamentales avec l'utilisateur : authentification via un code PIN, consultation du solde, dépôt et retrait d'argent. Cette simulation a pour but de familiariser les étudiants avec la manipulation des entrées/sorties, la gestion des conditions, la validation des données, et la tenue d'un historique simple.

Ce projet vise à développer les compétences suivantes :

- Analyse d'un besoin utilisateur simple et modélisation des fonctionnalités ;
- Programmation structurée en C++ avec gestion des conditions et boucles ;
- Conception modulaire avec fonctions et structures de données ;
- Validation et test des fonctionnalités avec gestion des cas d'erreurs ;
- Rédaction d'une documentation technique claire et complète.

1.1.Objectifs du projet

L'objectif principal est de développer une application console modulaire en C++ qui permettra :

- La saisie sécurisée d'un code PIN (avec un nombre limité d'essais) ;
- La consultation du solde du compte bancaire ;
- La gestion des opérations de dépôt et de retrait avec vérification des fonds disponibles ;
- La sauvegarde et la consultation de l'historique des opérations (en option) ;
- L'enregistrement des transactions pour assurer une traçabilité minimale.

1.2.Choix techniques

Le programme sera développé en langage C++ selon une approche itérative, permettant une montée en compétences progressive sur les notions clés de la programmation : gestion des entrées/sorties, conditions (if/else), boucles, fonctions, structures de données simples, et éventuellement fichiers pour la persistance des données.

L'interface utilisateur sera en mode console, privilégiant la simplicité et la clarté dans l'affichage, avec des messages explicites pour guider l'utilisateur et gérer les erreurs de saisie ou les cas limites (ex : dépassement du nombre d'essais, fonds insuffisants).

Exemples de scénarios d'utilisation

Exemple 1 – Authentification et consultation du solde :

L'utilisateur saisit son code PIN. En cas d'erreur, il dispose de trois essais. Une fois authentifié, il peut consulter son solde affiché à l'écran.

Exemple 2 – Dépôt et retrait d'argent :

L'utilisateur choisit de déposer 200 € sur son compte, puis retire 50 €. Le programme vérifie que le retrait est possible et met à jour le solde. Un récapitulatif est affiché à chaque opération.

Exemple 3 – Historique des opérations (optionnel) :

L'utilisateur peut consulter la liste des opérations précédentes, avec date, type d'opération, montant et solde après transaction.

1.3.Périmètre initial et évolutif

Dans un premier temps (itérations 1 à 4), l'application permettra de :

- Saisir un code PIN et limiter les essais ;
- Consulter et afficher le solde du compte ;
- Réaliser des opérations de dépôt et de retrait simples avec contrôle des règles métier ;
- Organiser le programme avec des fonctions et une structure claire.

Dans un second temps (itérations 5 et 6), des fonctionnalités supplémentaires seront introduites :

- Gestion de l'historique des opérations avec sauvegarde en mémoire ou fichier ;
- Gestion d'erreurs plus fine et messages d'aide à l'utilisateur ;
- Optimisation et modularisation avancée du code.

Cette simulation d'un guichet automatique bancaire constitue un excellent exercice pour mettre en pratique des concepts fondamentaux de la programmation, dans un contexte réaliste et concret. En plus de l'aspect technique, le projet sensibilise à l'importance de la robustesse du code, de la prise en compte des besoins utilisateurs et de la progression par étapes dans le développement logiciel.

1.4.Iterations :

1.4.1. Iteration 1 :

Permettre à l'utilisateur de s'authentifier en saisissant un code PIN. La sécurité est basique, avec un maximum de 3 tentatives pour entrer le bon code.

Compétences développées :

- Saisie sécurisée (via console)
- Utilisation des conditions (if/else)
- Boucle while pour limiter les essais
- Messages d'erreur adaptés

Fonctionnalités :

- Demande de saisie du code PIN
- Vérification immédiate de la saisie
- Message « Accès autorisé » ou « Accès refusé »
- Blocage après 3 essais incorrects

Exemple d'utilisation :

Entrez votre code PIN : 1234

Code incorrect, il vous reste 2 essais.

Entrez votre code PIN : 0000

Code incorrect, il vous reste 1 essai.

Entrez votre code PIN : 2580

Accès autorisé.

1.4.2. Itération 2 — Consultation du solde et affichage

Offrir la possibilité à l'utilisateur de consulter son solde bancaire actuel.

Compétences développées :

- Affichage formaté (cout)
- Gestion de variables numériques (float/double)
- Organisation simple du programme en fonctions

Fonctionnalités :

- Affichage clair du solde actuel
- Menu avec options numérotées
- Retour au menu principal après consultation

Exemple d'utilisation :

Menu principal :

1. Consulter solde
2. Déposer de l'argent
3. Retirer de l'argent
4. Quitter

Choix : 1

Votre solde actuel est de : 1523.75 €

1.4.3. Itération 3 — Dépôt d'argent avec validation

Permettre à l'utilisateur d'effectuer un dépôt d'argent, avec vérification du montant saisi.

Compétences développées :

- Validation de données (montants positifs)
- Mise à jour de variables
- Gestion d'erreurs liées aux entrées utilisateur

Fonctionnalités :

- Saisie d'un montant à déposer
- Vérification que le montant est supérieur à zéro
- Mise à jour du solde
- Message confirmant la transaction

Exemple d'utilisation :

Entrez le montant à déposer : -50

Montant invalide. Veuillez saisir un montant positif.

Entrez le montant à déposer : 250

Dépôt accepté. Nouveau solde : 1773.75 €

1.4.4. Itération 4 — Retrait d'argent avec contrôle de solde

Permettre à l'utilisateur de retirer de l'argent, avec contrôle du solde disponible pour éviter les découverts.

Compétences développées :

- Conditions combinées (vérification solde suffisant)
- Messages d'erreur précis
- Mise à jour du solde après retrait

Fonctionnalités :

- Saisie du montant à retirer
- Vérification que le solde est suffisant
- Refus du retrait si solde insuffisant
- Confirmation ou message d'erreur

Exemple d'utilisation :

Entrez le montant à retirer : 2000

Solde insuffisant. Votre solde est de 1773.75 €

Entrez le montant à retirer : 500

Retrait accepté. Nouveau solde : 1273.75 €

1.4.5. Itération 5 — Historique simple des opérations (mémoire)

Enregistrer en mémoire les opérations réalisées durant la session et les afficher à la demande.

Compétences développées :

- Utilisation de tableaux ou structures pour stocker des données
- Parcours et affichage des listes
- Gestion de la mémoire temporaire en session

Fonctionnalités :

- Enregistrement des opérations (type : dépôt ou retrait, montant)
- Affichage de l'historique des opérations
- Limitation de l'historique (exemple : les 10 dernières opérations)

Exemple d'utilisation :

Historique des opérations :

1. Dépôt : 250.00 €

2. Retrait : 100.00 €

3. Dépôt : 500.00 €

1.4.6. Itération 6 — Sauvegarde et chargement des données (fichiers)

Permettre la persistance des données entre les sessions grâce à la lecture et écriture dans des fichiers.

Compétences développées :

- Gestion des fichiers (fstream)
- Lecture/écriture formatée
- Gestion des erreurs d'accès aux fichiers

Fonctionnalités :

- Sauvegarde du solde et de l'historique en fin de session dans un fichier texte
- Chargement automatique des données au démarrage
- Messages d'erreur en cas de problème d'accès

Exemple d'utilisation :

Au démarrage :

Chargement des données en cours...

Solde actuel : 1273.75 €

Historique chargé : 3 opérations

1.4.7. Itération 7 — Interface utilisateur améliorée et gestion des erreurs

Rendre l'application plus conviviale, robuste, et intuitive avec une meilleure interface et gestion des erreurs.

Compétences développées :

- Modularisation du code (fonctions dédiées)
- Validation complète des entrées
- Messages d'erreur détaillés
- Amélioration de la présentation console

Fonctionnalités :

- Menu interactif clair avec choix numérotés
- Gestion des erreurs et saisies invalides (lettres, symboles)
- Message d'aide disponible
- Option de sortie propre du programme avec sauvegarde automatique

Exemple d'utilisation :

Menu principal :

1. Consulter solde
2. Déposer de l'argent
3. Retirer de l'argent
4. Historique
5. Quitter

Choix : a

Entrée invalide. Veuillez saisir un nombre entre 1 et 5.

Projet 3: Système de billetterie pour événements

Dans le cadre de ce projet, nous abordons la conception d'un système de billetterie destiné à un organisateur d'événements (concerts, matchs, spectacles). Actuellement, la gestion des ventes de billets se fait souvent de manière manuelle ou via des outils peu adaptés, ce qui peut engendrer des erreurs dans le suivi des places disponibles et des recettes.

L'objectif est donc de développer une application simple qui permette de gérer efficacement les événements et la vente des billets associés, afin d'assurer un suivi précis et une meilleure organisation.

Ce projet permet de travailler plusieurs compétences clés en programmation :

- Manipulation de données numériques (quantités, prix) et de dates simples (format texte) ;
- Organisation et structuration des données à l'aide de structures ou classes C++ ;
- Gestion des entrées/sorties sur fichiers pour la sauvegarde et la restauration des données ;
- Développement modulaire et validation des données saisies (contrôle des quantités, disponibilité).

1.1.Objectifs du projet

Ce projet vise à créer une application modulaire capable de :

- Ajouter de nouveaux événements en enregistrant des informations essentielles (nom, date, prix, nombre de places disponibles) ;
- Permettre la réservation d'une ou plusieurs places pour un événement donné, en mettant à jour automatiquement la disponibilité ;
- Afficher la liste des événements encore ouverts à la vente ;
- Calculer et afficher le total des ventes réalisées pour tous les événements.

1.2.Choix techniques

L'application sera développée en C++ et fonctionnera en mode console. L'accent sera mis sur :

- La manipulation simple des données numériques et textuelles, incluant la gestion basique des dates ;
- La structuration des données via des structures ou classes adaptées pour représenter les événements et leurs caractéristiques ;
- La lecture et l'écriture dans des fichiers pour sauvegarder l'état des événements et des ventes, assurant la persistance des données entre les sessions.

1.3.Exemples de scénarios d'utilisation

Exemple 1 – Ajout d'un événement :

L'organisateur saisit les informations suivantes :

- Nom : « Concert Rock »
- Date : 25/12/2025
- Prix du billet : 45 €
- Places disponibles : 300

Le système crée un nouvel événement accessible pour la réservation.

Exemple 2 – Réservation de billets :

Un client souhaite réserver 4 places pour le « Concert Rock ». Le programme vérifie la disponibilité, déduit les billets réservés du stock, et confirme la réservation :

4 billets réservés pour « Concert Rock ». Places restantes : 296.

Exemple 3 – Affichage des événements disponibles :

Le système affiche tous les événements avec leur date, prix et places encore disponibles :

1. Concert Rock – 25/12/2025 – 45 € – 296 places disponibles
2. Match de football – 01/01/2026 – 30 € – 150 places disponibles

Exemple 4 – Affichage des ventes totales :

Le responsable peut consulter le chiffre d'affaires généré par la billetterie :

Total des ventes : 7200 €

1.4.Iterations:

1.4.1. Itération 1 — Gestion basique d'événements

Objectifs :

- Apprendre à manipuler les entrées/sorties basiques en C++.
- Créer une structure simple pour représenter un événement (nom, date, prix).
- Saisir et afficher un événement.

Fonctionnalités :

- Saisie manuelle des informations d'un événement.
- Affichage formaté de cet événement.

Exemple d'utilisation :

L'utilisateur saisit :

- Nom : « Concert Jazz »
- Date : « 12/05/2026 »

- Prix : 35 €

Le programme affiche :

Événement ajouté : Concert Jazz, le 12/05/2026, prix du billet : 35 €

1.4.2. Itération 2 — Gestion des places disponibles et réservation simple

Objectifs :

- Ajouter un champ pour le nombre de places disponibles.
- Implémenter la réservation de billets avec mise à jour du stock.
- Vérifier que la réservation ne dépasse pas les places restantes.

Fonctionnalités :

- Ajout du nombre de places disponibles lors de la création d'un événement.
- Réservation d'un nombre donné de billets avec vérification de disponibilité.
- Message d'erreur si la demande dépasse le stock.

Exemple d'utilisation :

Saisie d'un événement avec 100 places.

Réservation de 5 billets valide :

5 billets réservés pour Concert Jazz. Places restantes : 95

Réservation de 200 billets :

Erreur : nombre de billets demandé supérieur aux places disponibles.

1.4.3. Itération 3 — Gestion d'une liste d'événements

Objectifs :

- Stocker plusieurs événements dans une collection (tableau ou vecteur).
- Afficher la liste des événements disponibles.

Fonctionnalités :

- Ajout multiple d'événements.
- Affichage dynamique de tous les événements avec leurs détails (nom, date, prix, places).

Exemple d'utilisation :

Affichage :

1. Concert Jazz – 12/05/2026 – 35 € – 95 places

2. Match de foot – 20/06/2026 – 50 € – 200 places

1.4.4. Itération 4 — Sauvegarde et chargement des données

Objectifs :

- Apprendre à lire et écrire des données dans des fichiers texte.
- Permettre la sauvegarde et la restauration de la liste d'événements.

Fonctionnalités :

- Écriture des événements dans un fichier à la fin du programme.
- Lecture des événements au démarrage pour restaurer l'état.

Exemple d'utilisation :

L'utilisateur crée 2 événements puis quitte le programme.

Au redémarrage, les événements sont automatiquement rechargés et affichés.

1.4.5. Itération 5 — Amélioration de la réservation et gestion des erreurs

Objectifs :

- Implémenter des contrôles plus robustes sur les entrées utilisateur.
- Gérer les cas d'erreurs (saisie incorrecte, réservation négative).
- Confirmation explicite après chaque réservation.

Fonctionnalités :

- Validation des entrées numériques (prix, nombre de billets).
- Message d'erreur clair pour toute saisie invalide.
- Confirmation avec récapitulatif après chaque réservation.

Exemple d'utilisation :

Réservation avec saisie de lettres au lieu de chiffres :

Entrée invalide, veuillez saisir un nombre entier positif.

1.4.6. Itération 6 — Calcul et affichage des ventes totales

Objectifs :

- Suivre le total des recettes générées par la vente de billets.
- Afficher le chiffre d'affaires global à la demande.

Fonctionnalités :

- Calcul automatique du montant des ventes après chaque réservation.
- Option d'affichage du total cumulé des ventes pour tous les événements.

Exemple d'utilisation :

Après plusieurs réservations :

Total des ventes : 6500 €

1.4.7. Itération 7 — Extension : Annulation et modification de réservation

Objectifs :

- Permettre d'annuler ou modifier une réservation existante.
- Mettre à jour les places disponibles et les ventes en conséquence.

Fonctionnalités :

- Interface simple pour annuler une réservation partielle ou totale.
- Modification des quantités réservées.
- Mise à jour dynamique des stocks et des totaux.

Exemple d'utilisation :

Le client annule 2 billets sur une réservation initiale de 5 :

Réservation modifiée. Places disponibles : 97. Total ventes ajusté.

IV. Grille d'évaluation

Itération 1 — Interaction simple— /10 points

Élément de compétence : 1. Analyser le problème ou l'opportunité	Poids
Critères de performance	
1.1 Collecter les exigences	2
1.4 Déterminer les données d'entrée/sortie	1
1.5 Choisir un algorithme simple (if/else)	1
Critères de performance	1
Élément de compétence : 3. Traduire l'algorithme en C++	
Critères de performance	
3.1 Organisation logique des instructions	1
3.2 Respect de la syntaxe	1
3.3 Cohérence avec l'algorithme	1
Élément de compétence : 4. Déboguer le code	
Critères de performance	
4.6 Bon fonctionnement du programme	2

Itération 2 — Boucles (for, while) — /14 points

Élément de compétence : 2. Produire l'algorithme	Poids
Critères de performance	
2.2 Déterminer la séquence logique	3
2.3 Vérification de l'exactitude de l'algorithme	2
Critères de performance	
Élément de compétence : 4. Déboguer le code	
4.6 Fonctionnement correct avec boucles	3
4.2 Repérage complet des erreurs	3

Itération 3 — Fonctions et modularisation — /14 points

Élément de compétence : 2. Produire l'algorithme	Poids
Critères de performance	
2.4 Représentation des algorithmes (modulaire)	4
Élément de compétence : 8. Produire la documentation	
8.2 Documentation des fonctions	3

Itération 4 — Tableaux pour gérer les menus — /14 points

Élément de compétence : 1. Analyser le problème	Poids
Critères de performance	
1.4 Définir les entrées/sorties (menu, prix)	2
Élément de compétence : 4. Écrire le programme (KP55)	
Critères de performance	
4.3 Utilisation optimale des tableaux	3
4.2 Programmation modulaire	2

Itération 5 — Structures (struct) pour plats/commandes — /14 points

Élément de compétence : 1. Analyser le problème	Poids
Critères de performance	
1.3 Identifier les besoins (champs des structs)	7
Élément de compétence : 6. Tester le programme	
Critères de performance	
6.2 Programmation appropriée des tests	7

Itération 6 — Modifications, annulations, remises — /14 points

Critères de performance	Poids
Élément de compétence : 6. Tester le programme	
6.1 Définir les cas de tests	2
6.3 Examen du code et des erreurs	5
Élément de compétence : 7. Optimiser le programme	
Critères de performance	
7.1 Réglage de la performance	7

Itération 7 — Gestion des clients et historique des commandes — /18 points

Critères de performance	
Élément de compétence : 1. Analyser le problème	Poids
1.2 Décomposer le problème	
Élément de compétence : 4. Écrire le programme	6
Critères de performance	
4.3 Utilisation optimale du langage	6
Critères de performance	
Élément de compétence : 6. Tester le programme	
6.5 Suivi des erreurs / amélioration	6

GRILLE DE CORRECTION DE LA LANGUE

**Les points soustraits, ci-dessous, peuvent être modifiés pour s'accorder aux choix de l'équipe-programme, concernant la pénalité accordée à la correction de la langue. Effacez cette phrase si vous utilisez cette grille.*

Communication claire	Communication le plus souvent claire	Communication floue	Communication difficile
-0	-0,5	-1,5	-2
Utilisation d'un vocabulaire précis et varié	Utilisation d'un vocabulaire précis	Utilisation d'un vocabulaire imprécis	Utilisation d'un vocabulaire inapproprié
-0	-0,5	-1,5	-2
Respect des normes (type de travail)	Respect de la plupart des normes (type de travail)	Non-respect des normes (type de travail)	Situation de communication inappropriée (type de travail)
-0	-0,5	-1,5	-2
Respect du code linguistique (≤2 fautes /page)	Respect de la plupart des règles du code linguistique (3-7 fautes /page)	Respect de la plupart des règles du code linguistique (8-10 fautes /page)	Respect de la plupart des règles du code linguistique (>10 fautes /page)
-0	-0,5 -2.5	-2.5 – 3.5	-4